hashlock.

Security Audit

Arcane

April 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Arcane team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Arcane is a Web3 privacy-focused infrastructure project building confidential and compliance-oriented blockchain solutions for decentralized finance and cross-chain applications. The protocol focuses on enabling private transactions, shielded asset transfers, and privacy-preserving DeFi interactions through advanced cryptographic mechanisms and decentralized infrastructure. Arcane's ecosystem includes components such as private liquidity infrastructure, privacy-enhanced wallets, cross-chain interoperability mechanisms, and institutional-grade compliance tooling designed to support secure and confidential on-chain activity across multiple blockchain ecosystems.

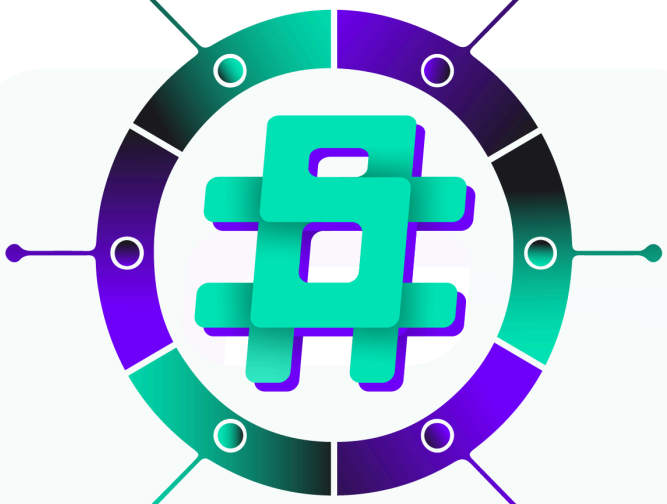
Project Name: Arcane

Project Type: DeFi

Website: https://arcaneprivacy.com/

Logo:

Visualised Context:



Project Name
Arcane

Launch Date
17/05/2026

Compiler Version
N/A

Language
RUST

Network
SOLANA

Token Ticker
TBA

Audit Scope

We at Hashlock audited the Rust code within the Arcane project; the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Arcane Program
Platform	Solana / Rust
Audit Date	April, 2026
Contract 1	programs/arcane/src/processor/add_pool.rs
Contract 2	programs/arcane/src/processor/backup_notes.rs
Contract 3	programs/arcane/src/processor/common.rs
Contract 4	programs/arcane/src/processor/deposit.rs
Contract 5	programs/arcane/src/processor/init_network.rs
Contract 6	programs/arcane/src/processor/init_notes.rs
Contract 7	programs/arcane/src/processor/mod.rs
Contract 8	programs/arcane/src/processor/update_network.rs
Contract 9	programs/arcane/src/processor/withdraw.rs
Contract 10	programs/arcane/src/state/commitment.rs
Contract 11	programs/arcane/src/state/mod.rs
Contract 12	programs/arcane/src/state/network_state.rs
Contract 13	programs/arcane/src/state/notes.rs
Contract 14	programs/arcane/src/state/nullifier.rs
Contract 15	programs/arcane/src/state/pool.rs
Contract 16	programs/arcane/src/state/poseidon_merkle_tree.rs

Contract 17	programs/arcane/src/error.rs
Contract 18	programs/arcane/src/events.rs
Contract 19	programs/arcane/src/lib.rs
Contract 20	programs/arcane/src/verifying_key.rs
Audited GitHub Commit Hash	5d22ec0ff734df8dc16feeb6fc993d2ac3f5460d
Fix Review GitHub Commit Hash	

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Medium severity vulnerability

13 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
cprograms/arcane/ - Deposit fixed-denomination SOL into a pool, recording a Poseidon commitment in an incremental Merkle tree.	**Contract achieves this functionality.**

- Allows relayers (signers) to Submit withdrawals on behalf of users and collect a configurable fee - Allows admins to initialize and update network configuration (fee-recipient wallets, fee splits). - Add new pools at distinct denominations.	
programs/relayer-registry/ - Stake the configured token to register as a relayer. - Unregister and recover the full stake after a fixed lock period. - Allows admin to initialize the per-mint global config and update minimum_stake and minimum_balance thresholds.	**Contract achieves this functionality.**
circuits/withdraw.circom - Verifies that the user knows a `(nullifier, secret)` pair whose Poseidon commitment is in the pool's Merkle tree, and binds (root, nullifierHash, recipient, relayer, fee, refund) as public inputs for on-chain settlement.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Arcane project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Arcane project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description

High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significanc	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.

Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] arcane#withdraw - Refund parameter accepted, bound, but never transferred

Description

The `withdraw` instruction accepts a `refund` parameter, packs it into the ZK proof's public inputs, and deducts it from the recipient's payout calculation, but never transfers the refund amount to any account. The funds are silently retained in the treasury PDA, creating a fund leak where the refund value is neither credited to the recipient nor sent to any other intended party.

Vulnerability Details

In `programs/arcane/src/processor/withdraw.rs`, the `refund` parameter flows through the function as follows:

1. Line 18: `refund: u64` is accepted without validation
2. Line 38: `public_inputs[5][24..].copy_from_slice(&refund.to_be_bytes());` — the refund is bound into the ZK proof
3. Line 48: `let recipient_amount = denomination - fee - refund;` — the refund is deducted from what the recipient receives
4. Lines 49–59: The treasury transfers `recipient_amount` (which excludes the refund) to the recipient
5. Lines 61–112: Fee distribution logic handles the fee split among relayers and wallets, but the refund is never referenced again

After these transfers complete, the refund amount remains in the treasury with no subsequent transfer instruction to emit it. The public event emitted at line 115–121 records only `denomination`, `fee`, and other fields—the refund is not logged.

This mirrors the Solana-equivalent scenario in Tornado's ETH instance. In `tornado-core/contracts/ETHTornado.sol`, line 37 enforces `require(_refund == 0, "Refund value is supposed to be zero for ETH instance");` because the refund parameter (designed for ERC20 instances to reimburse gas) is meaningless for a single-denomination (ETH/SOL) mixer. Arcane, being SOL-only, should apply the same constraint.

Impact

Any user who specifies a non-zero refund value is silently defrauded of that amount. The refund is deducted from their payout but never transferred to them or any other recipient. In a real deployment:

- A user intending to receive their full denomination less fees could lose any portion they allocate to the refund field.
- If the front-end or relayer software allows user-controlled refund input, an attacker or negligent user can drain themselves or others.
- The treasury account accumulates unclaimed, orphaned lamports that cannot be recovered by any legitimate withdrawal mechanism.
- Over thousands of withdrawals, even small refunds (e.g., 1 lamport per withdrawal) accumulate into a meaningful loss.

For a SOL-only mixer, the refund parameter has no legitimate use case and should be zero.

Recommendation

Add a require check to enforce that the refund must be zero:

```
pub fn handle_withdraw<'info>(
    ctx: Context<'_, '_, 'info, 'info, Withdraw<'info>>,
    proof: [u8; 256],
    root: [u8; 32],
    nullifier_hash: [u8; 32],
    fee: u64,
    refund: u64,
```

```
) -> Result<()> {
    let denomination = ctx.accounts.pool.denomination;
    require!(fee < denomination, Error::InsufficientAmount);
    require!(refund == 0, Error::InvalidRefund);
    // ... rest of function
```

Add the error variant to the error module:

```
#[error("Refund must be zero for SOL-only mixer")]
InvalidRefund,
```

Once the requirement is in place, the `refund` term at line 48 can be simplified to:

```
let recipient_amount = denomination - fee;
```

Status

Resolved

Low

[L-01] arcane#init_network - Permissionless initialization can be frontrun at deployment

Description

The InitNetwork instruction requires only a payer Signer and initializes a program-derived account (PDA) seeded with a constant value. Since the PDA is deterministic and single-shot, the first caller to invoke init_network becomes the network admin with an arbitrary choice of config_authority and wallets. Unlike Tornado Cash, which atomically initializes configuration in the contract constructor, Arcane's separation of deployment from initialization introduces a race condition window.

Vulnerability Details

The InitNetwork context constraint (programs/arcane/src/lib.rs:79-92) specifies:

```
#[derive(Accounts)]
pub struct InitNetwork<'info> {
    #[account(mut)]
    pub payer: Signer<'info>,
    #[account(
        init,
        payer = payer,
        space = 8 + NetworkState::SIZE,
        seeds = [CONFIG_ACCOUNT_SEED],
        bump,
    )]
    pub network_state: Account<'info, NetworkState>,
    pub system_program: Program<'info, System>,
}
```

The processor (programs/arcane/src/processor/init_network.rs:7-15) directly sets the config without any access control:

```
pub fn handle_init_network(
    ctx: Context<InitNetwork>,
```

```
    config_authority: Pubkey,
    wallets: Vec<Wallet>,
) -> Result<()> {
    let state_acc = &mut ctx.accounts.network_state;
    state_acc.config = new_config(config_authority, wallets)?;

    Ok(())
}
```

The PDA is seeded with the constant ["arcane-config"] (programs/arcane/src/lib.rs:15), making it unique and deterministic. Since the init anchor constraint enforces the account does not exist initially, only the first transaction succeeds. Any third party can frontrun the intended deployer's initialization call and seize control.

Impact

An attacker monitoring the blockchain can frontrun the deployment and invoke init_network before the legitimate deployer, gaining admin privileges. This allows arbitrary configuration of the config_authority and wallets, potentially redirecting protocol control or funds.

Recommendation

Populate the migrations/deploy.ts script to invoke init_network atomically as part of deployment, OR add a hardcoded address constraint to the payer Signer:

```
#[derive(Accounts)]
pub struct InitNetwork<'info> {
    #[account(mut, address = DEPLOY_AUTHORITY)]
    pub payer: Signer<'info>,
    // ... rest of accounts
}
```

Status

Acknowledged

[L-02] relayer-registry#initialize_config - Permissionless initialization can be frontrun at deployment

Description

The initialize_config instruction in the relayer-registry program exhibits the same permissionless initialization vulnerability as L-01. The instruction requires only an admin Signer and creates a program-derived account seeded deterministically. The first caller becomes the registry admin, with control over the minimum stake and other critical parameters. Notably, the same codebase correctly enforces access control in the update_config function, demonstrating that the team has the primitive but missed its application to initialization.

Vulnerability Details

The InitializeConfig context (programs/relayer-registry/src/lib.rs:254-277) specifies:

```
#[derive(Accounts)]
pub struct InitializeConfig<'info> {
    #[account(mut)]
    pub admin: Signer<'info>,
    #[account(
        init,
        payer = admin,
        space = 8 + GlobalConfig::INIT_SPACE,
        seeds = [SEED_GLOBAL_CONFIG, mint.key().as_ref()],
        bump
    )]
    pub config: Account<'info, GlobalConfig>,
    // ... additional accounts
}
```

The handler (programs/relayer-registry/src/lib.rs:24-34) sets admin to the transaction signer without validation:

```
pub fn initialize_config(ctx: Context<InitializeConfig>, minimum_stake: u64) ->
Result<()> {
    let config = &mut ctx.accounts.config;
```

```
    config.admin = ctx.accounts.admin.key();
    config.mint = ctx.accounts.mint.key();
    config.minimum_stake = minimum_stake;
    config.minimum_balance = 0;
    config.relayer_count = 0;

    Ok(())
}
```

In contrast, the update_config function (line 405+) correctly gates mutations with `address = config.admin`, proving the pattern is known to the team.

Impact

An attacker can frontrun initialization and claim admin privileges over the relayer registry, controlling minimum stake requirements and other configuration parameters.

Recommendation

Apply the same access-control pattern used in update_config to initialize_config:

```
#[derive(Accounts)]
pub struct InitializeConfig<'info> {
    #[account(mut, address = DEPLOY_AUTHORITY)]
    pub admin: Signer<'info>,
    // ... rest of accounts
}
```

Alternatively, populate the deployment script to invoke initialize_config atomically.

Status

Acknowledged

[L-03] withdraw.circom#MerkleTreeChecker - Path index selector not constrained to binary

Description

The Switcher template in the Merkle tree proof does not enforce that the path index selector signal is binary (0 or 1). This creates a soundness gap: a prover with unbounded sel values can construct alternative proof paths. However, because the underlying hash function (Poseidon) is cryptographically secure, computing a valid preimage still requires approximately 2^254 operations, making practical forgery infeasible.

Vulnerability Details

In circuits/withdraw.circom (lines 22-43), the MerkleTreeChecker template uses Switcher to select between left and right path elements:

```
template MerkleTreeChecker(levels) {
    signal input leaf;
    signal input root;
    signal input pathElements[levels];
    signal input pathIndices[levels];

    component switchers[levels];
    component hashers[levels];

    for (var i = 0; i < levels; i++) {
        switchers[i] = Switcher();
        switchers[i].sel <== pathIndices[i];
        switchers[i].L <== i == 0 ? leaf : hashers[i - 1].out;
        switchers[i].R <== pathElements[i];
        // ...
    }
    // ...
}
```

The Switcher template (`node_modules/circomlib/circuits/switcher.circom`) does not constrain sel to {0, 1}. Tornado Cash's equivalent DualMux template includes the binary constraint:

```
s * (1 - s) === 0;
```

Without this constraint, pathIndices[i] can take any value in the field, allowing multiple valid openings of the merkle path. However, forging a proof still requires finding a hash preimage, which is infeasible under Poseidon hardness (~2^254 work).

Impact

Theoretical soundness gap: the circuit permits multiple valid proof paths for a single commitment, violating strict zero-knowledge properties. Practical impact is negligible because Poseidon is assumed cryptographically hard. No known efficient forge exists.

Recommendation

Add explicit binary constraints to each pathIndices signal inside the MerkleTreeChecker loop:

```
for (var i = 0; i < levels; i++) {
    pathIndices[i] * (1 - pathIndices[i]) === 0;
    // ... rest of loop
}
```

After modification, recompile the circuit and regenerate the proving and verification keys (conditional on a trusted setup ceremony if keys are public).

Status

Acknowledged

[L-04] arcane#add_pool - Tree levels parameter accepts zero

Description

The add_pool instruction does not validate that the tree levels parameter is greater than zero. Passing levels=0 to PoseidonMerkleTree::new() causes a panic in the zeros() function, reverting the transaction. While no invalid state persists, the lack of explicit validation is inconsistent with reference implementations and represents a potential footgun for future changes.

Vulnerability Details

The add_pool instruction (programs/arcane/src/processor/add_pool.rs:5-11) accepts a levels parameter without range validation:

```
pub fn handle_add_pool(ctx: Context<AddPool>, denomination: u64, levels: u32) ->
Result<()> {
    let pool = &mut ctx.accounts.pool;
    pool.denomination = denomination;
    pool.merkle_tree = PoseidonMerkleTree::new(levels)?;

    Ok(())
}
```

The PoseidonMerkleTree::new() function (programs/arcane/src/state/poseidon_merkle_tree.rs:21-38) eventually calls zeros():

```
pub fn new(levels: u32) -> Result<Self> {
    require!(levels <= MAX_LEVELS as u32, Error::InvalidLevels);

    let filled_subtrees: Vec<[u8; 32]> = (0..levels).map(zeros).collect();

    let mut roots = [[0u8; 32]; ROOT_HISTORY_SIZE];
    roots[0] = zeros(levels - 1);
    // ...
}
```

When levels=0, the line roots[0] = zeros(0u32 - 1) underflows. With overflow-checks=true (default in debug), this causes a panic. Without overflow checks, the subtraction wraps to 0xFFFFFFFF, triggering the panic! ("Index out of bounds") in

the zeros() function (line 189).

Tornado Cash's MerkleTreeWithHistory.sol includes explicit validation: require(_levels > 0, ...).

Impact

Calling add_pool with levels=0 causes the transaction to revert with a panic, preventing pool creation. No bad state is persisted, but the error is not handled gracefully.

Recommendation

Add explicit validation to the add_pool handler or to PoseidonMerkleTree::new():

```
pub fn new(levels: u32) -> Result<Self> {
    require!(levels > 0 && levels <= MAX_LEVELS as u32, Error::InvalidLevels);
    // ...
}
```

This provides a clear error message and aligns with standard Merkle tree initialization patterns.

Status

Resolved

[L-05] arcane#new_config - Wallet count limit allows only 4 wallets instead of 5

Description

The wallet count check uses a strict less-than comparison against `MAX_WALLETS`, so the effective maximum is 4, even though the constant is set to 5 and the account layout reserves space for 5 slots.

Vulnerability Details

File: `programs/arcane/src/processor/common.rs:13`

```
require!(wallets.len() < MAX_WALLETS, Error::TooManyWallets);
```

With `MAX_WALLETS = 5` (`programs/arcane/src/state/network_state.rs:3`), this rejects any input where `wallets.len() == 5`.

The `NetworkState::SIZE` calculation at `state/network_state.rs:25` reserves space for 5 wallets, confirming the intended cap. The off-by-one is in the validation operator, not the storage allocation.

Impact

Administrators cannot configure the documented maximum of 5 fee-split recipients. This is a usability constraint that limits operational flexibility; no security impact.

Recommendation

Change the comparison operator from `<` to `<=`:

```
require!(wallets.len() <= MAX_WALLETS, Error::TooManyWallets);
```

Status

Resolved

[L-06] PoseidonMerkleTree#is_known_root - Empty-tree root remains accepted indefinitely

Description

The Merkle tree's root-verification function does not reject the empty-tree root after the tree has been populated with deposits. The zero-filled initialization root survives because it is non-zero in Poseidon output space, bypassing the global zero-check. Though exploitation requires preimage-inverting a secure hash function—currently infeasible—defense-in-depth suggests excluding this root after deposits begin.

Vulnerability Details

File: programs/arcane/src/state/poseidon_merkle_tree.rs line 29

```
roots[0] = zeros(levels - 1)  // Non-zero Poseidon output; survives checks
```

File: programs/arcane/src/state/poseidon_merkle_tree.rs line 74

```
if root == [0; 32] {
    return false;  // Only rejects true-zero bytes; does not reject zeros(levels-1)
}
```

This design matches the reference Tornado Cash implementation (tornado-ref/contracts/MerkleTreeWithHistory.sol:47), where the empty-tree root is similarly stored and persists as valid. An attacker would need to compute a valid Poseidon preimage of zeros(0), which is not the empty-tree root but rather a different value—currently computationally infeasible.

Impact

If Poseidon's preimage resistance is compromised in the future, an attacker could forge withdrawals using a crafted root without actually depositing. The current risk is theoretical and depends on breaking cryptographic assumptions. In practice, this does not affect the security of the ongoing deployment.

Recommendation

Implement defense-in-depth by skipping the initialization root once deposits have been made:

```
pub fn is_known_root(&self, root: &[u8; 32]) -> bool {
    if root == &[0; 32] {
        return false;
    }
    // Skip slot 0 (initialization root) once tree is non-empty
    let start_index = if self.next_index > 0 { 1 } else { 0 };
    self.roots[start_index..].contains(root)
}
```

Status

Acknowledged

[L-07] PoseidonMerkleTree#insert - Hash function panics on field-overflow input

Description

The tree insertion function uses an unwrap on a fallible Poseidon hash operation without proper error handling. The light-poseidon library rejects inputs larger than the BN254 field modulus by returning an error, which the current code converts into an opaque program failure. While honest protocol paths never produce such inputs, a malicious client could craft an out-of-field input to trigger an unhandled panic instead of a clean rejection.

Vulnerability Details

File: programs/arcane/src/state/poseidon_merkle_tree.rs lines 58–62

```
let hash = light_poseidon::Poseidon::hashv(&[left, right])
    .unwrap();  // Panics if input >= BN254 field modulus
```

The light-poseidon library returns `Err(InputLargerThanModulus)` when an input is not field-reduced. The `.unwrap()` call converts this into a transaction failure with error code `ProgramFailedToComplete`, obscuring the true reason. Honest deposits always produce field-safe inputs; however, a malicious client sending out-of-range data bypasses this check.

Impact

A malicious client can deliberately cause a program to panic rather than a graceful error rejection, resulting in a less informative failure message. This does not allow stealing funds or bypassing authorization, but it does degrade error reporting and could complicate operational debugging.

Recommendation

Replace the unwrap with explicit error handling:

```
let hash = light_poseidon::Poseidon::hashv(&[left, right])
    .map_err(|_| Error::HashFailed)?;
```

Add a new error variant to programs/arcane/src/error.rs (if not already present):

```
#[error("Poseidon hash operation failed")]
HashFailed,
```

Status

Resolved

[L-08] arcane#add_pool - Pool denomination accepts zero

Description

The pool creation function does not validate that the denomination (the minimum deposit amount) is non-zero. An administrator can create a pool with denomination=0, allowing deposits and withdrawals that transfer zero lamports. This causes unnecessary state bloat and violates the Tornado Cash parity design, where denominations must be positive.

Vulnerability Details

File: programs/arcane/src/processor/add_pool.rs lines 5–11

```
// No check that denomination > 0
pub fn add_pool(denomination: u64) {
    // Pool created with denomination = 0 is valid
}
```

A pool with denomination=0 allows deposits and withdrawals without transferring any SOL:

- Deposit: `transfer_amount = 0 * fee = 0 lamports` (no payment required)
- Withdrawal: `transfer_amount = 0 * fee = 0 lamports` (no payment received)

This is inconsistent with the Tornado Cash reference implementation (Tornado.sol:46):

```
require(_denomination > 0, "denomination should be greater than 0");
```

Impact

Administrators can inadvertently create unusable pools, and state accounts accumulate deposits and withdrawals with zero value transfer. This causes unnecessary on-chain storage consumption without providing protocol utility.

Recommendation

Add validation in the add_pool function:

```
require!(denomination > 0, Error::InvalidDenomination)?;
```

Add the new error variant to programs/arcane/src/error.rs (if not already present):

```
#[error("Pool denomination must be greater than zero")]
InvalidDenomination,
```

Status

Resolved

[L-09] arcane#withdraw - Fee distribution math can DoS withdrawals on misconfiguration

Description

The fee distribution mechanism applies a per-wallet split calculation that can exhaust the withdrawal fee if the administrator configures wallets with aggregate splits exceeding the fee amount. When this occurs, every withdrawal reverts, effectively blocking the protocol. This is a configuration error rather than a code bug, but the system lacks validation to prevent it.

Vulnerability Details

Files: programs/arcane/src/processor/withdraw.rs lines 73–93; programs/arcane/src/processor/common.rs lines 9–19; programs/arcane/src/state/network_state.rs line 4

The fee distribution logic:

```
for fee_split in &network_state.fee_splits {
    let transfer_amount = (denomination * fee_split) / 10000;
    remaining_fee = remaining_fee.checked_sub(transfer_amount)
        .ok_or(Error::InvalidFeeAmount)?;
}
```

If the sum of fee splits is configured such that:

```
Σ (denomination × split_i / 10000) > fee
```

Then `remaining_fee.checked_sub()` will fail for every withdrawal.

Example: If the relayer sets the fee = 50 bps (basis points) and the admin configures wallet splits totaling 110 bps with denomination D, the required per-withdrawal payout is `D × 110 / 10000 bps`, which exceeds the 50 bps fee, causing all withdrawals to revert with `InvalidFeeAmount`.

Impact

Misconfigured fee splits can entirely block withdrawals for all users of a pool until the configuration is corrected. This is a denial of service affecting protocol availability, though it requires explicit misconfiguration by the administrator.

Recommendation

Add a validation check in the new_config function (programs/arcane/src/processor/common.rs):

```
let total_split: u64 = network_state.fee_splits.iter().sum();
require!(
    total_split <= FEE_SPLIT_DENOMINATION,
    Error::InvalidWallets
)?;
```

Additionally, document that relayers must set fees such that:

```
fee >= (total_split × denomination) / 10000
```

Status

Acknowledged

[L-10] arcane - Treasury, commitment, and nullifier PDAs not scoped per pool

Description

The treasury, commitment, and nullifier program-derived accounts are derived using a global seed namespace rather than pool-specific seeds. This means all pools share the same treasury address, and commitment/nullifier accounts could collide across pools, preventing cross-pool reuse. While the security model is presently bounded by Poseidon hardness, defense-in-depth design suggests isolating each pool's accounts.

Vulnerability Details

Files: programs/arcane/src/lib.rs lines 173 (treasury), 175–181 (commitment), 204–209 (nullifier)

Treasury derivation uses a global seed with no pool identifier:

```
// lib.rs:173
seeds = [SEED_TREASURY],
```

Commitment and nullifier derivation similarly lack pool-specific scope:

```
// lib.rs:175-181 (commitment)
// lib.rs:204-209 (nullifier)
// No pool.key() or denomination in seed list
```

This means:
- Treasury (treasury PDA): All pools deposit and withdraw into the same account, combining SOL across pools.
- Commitment (leaf_commitment PDA): Two different pools could theoretically derive the same commitment address, though this is astronomically unlikely due to Poseidon collision resistance.
- Nullifier (nullifier_account PDA): Similar collision risk across pools.

Impact

The treasury design is operationally correct under the assumption of a single active pool, but multiple pools with distinct denominations or fee structures will have their

funds mixed. Commitment and nullifier collisions are cryptographically unlikely but represent an unmitigated attack surface if hash function assumptions degrade.

Recommendation

Include the pool public key (or denomination) in the seed list for each account. For treasury:

```
seeds = [SEED_TREASURY, pool.key().as_ref()],
```

For commitment:

```
seeds = [SEED_COMMITMENT, pool.key().as_ref(), &leaf_commitment_bytes],
```

For nullifier:

```
seeds = [SEED_NULLIFIER, pool.key().as_ref(), &nullifier_bytes],
```

Update the treasury signer derivation in programs/arcane/src/processor/withdraw.rs (line 45) to match the new seed format.

Status

Acknowledged

[L-11] relayer-registry#UpdateConfig - Config account lacks explicit seeds constraint

Description

The UpdateConfig instruction does not enforce a PDA seed constraint on the configuration account, unlike all other contexts in the relayer-registry module. The account is instead validated only by an address check (`address = config.admin`), which covers the current single-config deployment but represents a deviation from the module's seed-based account validation pattern and creates a potential footgun for future changes.

Vulnerability Details

File: programs/relayer-registry/src/lib.rs lines 404–409

```
#[derive(Accounts)]
pub struct UpdateConfig<'info> {
    #[account(mut)]
    pub config: Account<'info, GlobalConfig>,
    // Missing: seeds = [SEED_GLOBAL_CONFIG, mint.key().as_ref()]
    // Instead, relying only on:
    // address = config.admin
}
```

Contrast with other contexts in the same module, which explicitly validate seeds:

```
#[account(
    seeds = [SEED_GLOBAL_CONFIG, mint.key().as_ref()],
    bump
)]
pub config: Account<'info, GlobalConfig>,
```

Impact

While the address constraint provides safety for the current single-config case, the inconsistency introduces a maintenance risk. If the module evolves to support multiple configurations, the absence of seed validation could allow unauthorized configuration

updates or account confusion. This is a code hygiene issue rather than a critical vulnerability.

Recommendation

Add the explicit seeds constraint to UpdateConfig:

```
#[derive(Accounts)]
pub struct UpdateConfig<'info> {
    #[account(
        mut,
        seeds = [SEED_GLOBAL_CONFIG, mint.key().as_ref()],
        bump
    )]
    pub config: Account<'info, GlobalConfig>,
    pub mint: Account<'info, Mint>,
}
```

Ensure that all references to global configuration accounts throughout the module use consistent seed derivation.

Status

Resolved

[L-12] relayer-registry#add_stake - Adding stake does not extend lock period

Description

When a relayer adds additional stake to an already-registered account, the lock period is not refreshed. If the original lock has expired, the relayer can immediately unregister and withdraw all staked tokens, including the newly added amount. This allows circumventing the lock mechanism by making incremental stake additions.

Vulnerability Details

File: programs/relayer-registry/src/lib.rs lines 140–169

```
pub fn add_stake(ctx: Context<AddStake>, amount: u64) -> Result<()> {
    let relayer = &mut ctx.accounts.relayer;
    relayer.staked_amount = relayer.staked_amount.checked_add(amount).ok_or(...)?;
    // Missing: relayer.unlock_at = now + LOCK_PERIOD
    Ok(())
}
```

For comparison, the initial registration properly sets the lock (lines 55–57):

```
pub fn register_relayer(ctx: Context<RegisterRelayer>, ...) -> Result<()> {
    let relayer = &mut ctx.accounts.relayer;
    relayer.unlock_at = now + LOCK_PERIOD;
    // ...
}
```

Exploitation scenario:

1. Relayer registers with 9000 ARCN, unlock_at = t0 + LOCK_PERIOD
2. At time t0 + LOCK_PERIOD + 1, the lock expires
3. Relayer calls add_stake with 1000 ARCN, staked_amount becomes 10000 ARCN
4. unlock_at remains t0 + LOCK_PERIOD (expired)
5. Relayer immediately calls unregister_relayer and withdraws the full 10000 ARCN

Impact

A relayer can break the intended lock mechanism by adding stake after the original lock period expires. This reduces the security model's effectiveness: the stake is supposed to

be locked for a minimum duration to ensure relayer accountability, but this design flaw allows bypassing that guarantee.

Recommendation

Refresh the lock period when stake is added. At the end of the add_stake function (after line 165):

```
relayer.unlock_at = Clock::get()?.unix_timestamp + LOCK_PERIOD;
```

This matches the pattern in register_relayer (lines 55–57) and ensures that all stake contributions are subject to the full lock period.

Status

Acknowledged

[L-13] arcane#new_config - Default Pubkey accepted as configuration authority

Description

The network state configuration function does not reject Pubkey::default() as the authority, allowing an administrator to inadvertently set the configuration authority to a null key. This would render the network state immutable and unrecoverable. The relayer-registry module implements the equivalent check, but it is absent in the arcane module.

Vulnerability Details

File: programs/arcane/src/processor/common.rs
The new_config function does not validate the authority parameter:

```
pub fn new_config(authority: Pubkey, ...) {
    // No check for Pubkey::default()
    network_state.authority = authority;
}
```

In contrast, programs/relayer-registry/src/lib.rs (lines 223–225) implements the necessary safeguard:

```
require!(admin != Pubkey::default(), ErrorCode::InvalidAdmin);
```

Impact

If authority is set to Pubkey::default() (0x00...00), no valid signer can ever match this key, preventing any future updates to the network state. The configuration becomes permanently frozen, effectively bricking the protocol's ability to adjust fees, wallets, or other parameters. This is an operational mistake with permanent consequences.

Recommendation

Add a validation check in the new_config function immediately after accepting the authority parameter:

```
require!(authority != Pubkey::default(), Error::InvalidAuthority)?;
```

Add the new error variant to programs/arcane/src/error.rs (if not already present):

```
#[error("Configuration authority cannot be the default public key")]
InvalidAuthority,
```

Status

Resolved

[L-14] arcane#withdraw - Recipient pubkey sent to Range.org on every withdraw

Description

At withdraw time, the recipient's pubkey is sent to Range.org via an HTTPS query inside the Switchboard OracleJob. Every withdrawal becomes a logged entry at Range.org, tying the recipient to the act of receiving Arcane funds.

Vulnerability Details

File: programs/arcane/src/processor/withdraw.rs:31

```
verify_risk_score_feed(&mut verifier, &ctx.accounts.recipient.key())?;
```

The URL with the recipient embedded is constructed at common.rs:57-63:

```
let url = format!(
    "https://api.range.org/v1/risk/address?address={}&network=solana",
    addr_b58
);
```

Switchboard oracles fetch this URL when serving a quote. Range receives the recipient address in cleartext.

The deposit-side check at `deposit.rs:21` correctly uses `sender.key()`: privacy-respecting because the sender is already public as the tx signer. The withdraw-side check on the recipient is the regression.

Impact

Range.org logs every recipient address with a timestamp. Off-chain (Range query timestamp) correlates with on-chain (withdraw tx) within ~10 seconds, enabling

deanonymization, README still markets pure privacy; the compliance-gating regression is undocumented to users.

Recommendation

Delete withdraw.rs:31. The deposit-side check at deposit.rs:21 remains correct.

Status

Acknowledged

[L-15] arcane@deps - devnet feature flag bricks mainnet deployment

Description

The switchboard-on-demand dependency is built with the devnet feature flag. `default_queue()` resolves to the devnet queue pubkey. Combined with the Anchor `address = default_queue()` constraint at deposit and withdraw, every call reverts with `ConstraintAddress` on a mainnet deploy.

Vulnerability Details

File: programs/arcane/Cargo.toml:27

```
switchboard-on-demand = { version = "0.12.1", features = ["anchor", "devnet"] }
```

SDK selector at `switchboard-on-demand-0.12.1/src/utils.rs`:

```
pub fn is_devnet() -> bool {
    cfg!(feature = "devnet") || std::env::var("SB_ENV").unwrap_or_default() ==
"devnet"
}
```

`is_devnet()` also honors the `SB_ENV=devnet` env var at runtime — fixing Cargo.toml alone is not sufficient.

Impact

Total protocol DoS on mainnet deployment. No fund risk (failure is at validation, before any state mutation).

Recommendation

```
switchboard-on-demand = { version = "0.12.1", features = ["anchor"] }
```

For local devnet testing, gate behind a dev feature:

```
[features]
```

```
devnet = ["switchboard-on-demand/devnet"]
```

Ensure the mainnet deploy environment has `SB_ENV` unset.

Status

Resolved

[L-16] arcane#common - Hardcoded Ed25519 sigverify ix index 0 panics on mismatch

Description

The function reads the instruction at index 0 of the containing transaction via `verifier.verify_instruction_at(0).unwrap()`. If index 0 is not an Ed25519 sigverify instruction, the SDK's internal `assert!` panics. Two paths hit this: (a) tx without sigverify prepended, (b) tx with `ComputeBudgetProgram.setComputeUnitLimit` prepended: sigverify lands at index 1, `common.rs:31` still reads index 0, same panic.

Vulnerability Details

File: programs/arcane/src/processor/common.rs:31

```
let quote = verifier.verify_instruction_at(0).unwrap();
```

The panic fires inside the Switchboard SDK at `switchboard-on-demand-0.12.1/src/sysvar/ix_sysvar.rs:88`:

```
assert!(check_pubkey_eq(program_id, ED25519_PROGRAM_ID));
```

This is a raw `assert!`, not a `Result::Err`. Replacing `.unwrap()` with `?` would NOT fix this: the SDK panics before returning a Result.

Impact

Self-DoS. Caller's tx reverts; no funds move incorrectly, no state corruption, no other-user impact.

Recommendation

For the `ComputeBudget` collision, scan the instructions sysvar for the sigverify position rather than hardcoding index 0.

Status

Acknowledged

[L-17] arcane@admin - No protocol pause primitive

Description

The protocol has no admin pause mechanism. `Configuration` has no `paused` flag, and there is no `pause()` or `set_paused` instruction. Every operational issue requires the slow "diagnose → fix → upgrade → deploy" cycle instead of immediate halt.

Vulnerability Details

File: programs/arcane/Cargo.toml

```
pub struct Configuration {
    pub authority: Pubkey,
    pub wallets: Option<Vec<Wallet>>,
}
```

Impact

Slow recovery from any operational issue.

Recommendation

Add `paused: bool` to `Configuration`. Add admin-gated `set_paused` instruction. Add early-exit in deposit and withdraw handlers:

```
require!(!ctx.accounts.network_state.config.paused, Error::ProtocolPaused);
```

Status

Acknowledged

[L-18] arcane@deps - `groth16-solana` sourced from personal fork with `branch = master`

Description

The Groth16 proof verifier is pulled from a personal GitHub fork tracked by `branch = "master"` instead of a pinned commit. The fork owner can push arbitrary code to master at any time; Arcane's next build pulls it. The verifier runs on every withdrawal.

Vulnerability Details

File: programs/arcane/src/state/network_state.rs:12-16

```
groth16-solana = { git = "https://github.com/sunmoon11100/groth16-solana.git", branch = "master" }
```

Impact

Supply-chain exposure on the cryptographic gate.

Recommendation

Switch URL to the canonical upstream and pin to a specific commit:

```
groth16-solana = { git = "https://github.com/Lightprotocol/groth16-solana", rev = "<commit-sha>" }
```

Status

Resolved

QA

[Q-01] arcane#common - `quote.oracle_count` not enforced

Description

The SC accepts any quote with `sig_count >= 1`. Per Switchboard docs, the minimum oracle quorum is the consumer's responsibility via `numSignatures` off-chain and `quote.oracle_count` on-chain.

Vulnerability Details

`programs/arcane/src/processor/common.rs:38-42` never reads `quote.oracle_count`. `QueueAccountData` has no `min_signatures` field, the SDK delegates quorum enforcement entirely to the consumer.

Impact

None today under the realistic threat model. Would activate only if Switchboard supply-chain compromise occurs.

Recommendation

Optional defense-in-depth:

```
require!(quote.oracle_count >= MIN, Error::InsufficientQuorum);
```

Status

Acknowledged

[Q-02] arcane#common - `< 50` slot freshness check is unreachable dead code

Description

The check at `common.rs:36` requires quote age < 50 slots. Unreachable: the SDK already enforces `max_age = 30` inside `verify_instruction_at` before returning the quote.

Vulnerability Details

`programs/arcane/src/processor/common.rs:36`

```
require!(slot.saturating_sub(quote_slot) < 50, Error::StaleQuote);
```

Impact

None. Just unreachable code.

Recommendation

Delete the line, and optionally add `.max_age(30)` explicitly on the verifier builder for documentation.

Status

Acknowledged

Centralisation

The Arcane project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Arcane project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged.". Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au